

Proyecto No. 1: Diseño, Desarrollo y Consulta de Base de Datos

Avances de la Etapa 1

Objetivo

Procesar los archivos CSV proporcionados, diseñar una base de datos PostgreSQL coherente, automatizar su carga mediante Python e incorporar una ingesta adicional desde la API pública de la NBA.

Análisis inicial del conjunto de datos

Archivo o entidad	Registros detectados	Decisión
Game.csv	62,448 originales	Limpiar duplicados
Partidos únicos	62,379	Cargar en game
Temporadas 2015-2020	7,069 partidos	Cobertura mínima
Equipos	45 IDs actuales/históricos	Catálogo ampliado
Jugadores	8,996 normalizados	Incluye históricos del draft
Árbitros	143	Relación N:M con partidos
Salarios de jugador	1,292	Normalizar por temporada
News.csv	Aprox. 772 MB	Excluir inicialmente

Problemas de calidad identificados

- 69 identificadores de partido aparecían repetidos; se conserva la fila con mayor cantidad de valores disponibles.
- Algunos identificadores numéricos aparecen con terminación decimal y deben convertirse a enteros.
- Los salarios de equipos están distribuidos en columnas por temporada y deben transformarse a filas.
- Los salarios de jugadores no incluyen player_id, por lo que se aplica una asociación controlada por nombre normalizado.
- Team.csv contiene únicamente 30 equipos actuales, mientras que los partidos incluyen 45 IDs de franquicias.

Modelo de datos propuesto

Tabla	Función	Relaciones principales
team	Equipos y franquicias	player, game, salary, draft
season	Temporadas NBA	game, salary, API stats
player	Jugadores	team, salary, draft, API stats
game	Partidos y resultados	season, equipos, árbitros
official	Árbitros	game_official
game_official	Tabla asociativa	game N:M oficial
team_salary	Salario total	team y season
player_salary	Salario individual	player, team y season
draft_selection	Selecciones del draft	player y team
team_history	Cambios históricos	team
player_season_stat	Ingesta NBA API	player, team y season

Cardinalidades principales

- Temporada 1:N Partido; Equipo 1:N Partido como local y como visitante.
- Equipo 1:N Salario de equipo; Temporada 1:N Salario de equipo.
- Jugador 1:N Salario de jugador; Equipo 1:N Salario de jugador; Temporada 1:N Salario de jugador.
- Partido N:M Árbitro, resuelta mediante game_official.
- Jugador 1:N Estadística por temporada; Equipo 1:N Estadística por temporada.

Automatización desarrollada

El cargador de Python lee los CSV directamente desde Data.zip, completa los catálogos históricos, elimina duplicados, transforma los salarios y carga las tablas respetando el orden de las llaves foráneas. La validación en seco se ejecutó correctamente con el ZIP original.

Ingesta mediante NBA API

Se preparó una ingesta con el endpoint `LeagueDashPlayerStats`. El resultado se almacena en `player_season_stat` utilizando jugador, equipo y temporada como llave compuesta. Esto permite repetir la ejecución sin duplicar registros y aporta estadísticas de puntos, asistencias, rebotes, robos, bloqueos y porcentajes de tiro.

Evidencia lista

- Diagrama Entidad-Relación en formato Mermaid.
- Script `schema.sql` con tablas, restricciones e índices.
- Script `load_csv.py` para limpieza y carga de los CSV.
- Script `load_api.py` para la ingesta oficial.
- Consultas de verificación y consultas preliminares.
- README con instrucciones de instalación y ejecución en Windows.

Próximos pasos

- 1 Crear la base `nba_project` en PostgreSQL o iniciar Docker Compose.
- 2 Ejecutar `load_csv.py` con el ZIP y guardar evidencia de los conteos.
- 3 Ejecutar `verification.sql` y confirmar cero duplicados y registros huérfanos.
- 4 Ejecutar `load_api.py` para 2020-21 y verificar `player_season_stat`.
- 5 Exportar el diagrama ER como imagen y subir los avances a GitHub.

Estado actual: el diseño y los programas de la Etapa 1 están preparados; falta ejecutar la carga contra la instancia PostgreSQL del grupo y conservar capturas de los resultados como evidencia.